

Competitive Programming

Section 46 Editorial: Rerooting and Advanced Tree DP

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Sum Distances	4
2.1	Restatement	4
2.2	Algorithm and proof	4
2.3	Complexity	4
2.4	Full C++ solution	4
2.5	Full Python solution	4
3	B. Farthest Node Distance	6
3.1	Restatement	6
3.2	Algorithm and proof	6
3.3	Complexity	6
3.4	Full C++ solution	6
3.5	Full Python solution	6
4	C. Paths Through Node	8
4.1	Restatement	8
4.2	Algorithm and proof	8
4.3	Complexity	8
4.4	Full C++ solution	8
4.5	Full Python solution	8
5	Testing discussion	10

1 Editorial overview

1.1 Detailed solution guide

A. Sum of Distances from Every Root. First root the tree at 1. A postorder DFS computes $\text{sub}[v]$ and $\text{down}[v]$, where $\text{down}[v]$ is the sum of distances from v to vertices in its subtree. The answer for root 1 is $\text{down}[1]$. When rerooting across edge $v-c$, the $\text{sub}[c]$ vertices inside c 's subtree become one step closer, and the other $n - \text{sub}[c]$ vertices become one step farther. Therefore $\text{ans}[c] = \text{ans}[v] - \text{sub}[c] + (n - \text{sub}[c])$.

B. Tree Diameter. Run a DFS/BFS from any vertex to find a farthest vertex a . Run again from a ; the farthest vertex b gives the diameter distance. The first search reaches an endpoint of some diameter because any path can be extended toward a farthest vertex, and the second search measures the longest path starting from that endpoint.

C. Pairs Separated by Removing a Vertex. Removing vertex v splits the tree into components: one for each child subtree and, unless v is the root, one parent-side component of size $n - \text{sub}[v]$. Pairs whose path passes through v are exactly pairs chosen from two different components. Summing component sizes seen so far gives $\text{sum} += \text{seen} * \text{comp_size}$, then $\text{seen} += \text{comp_size}$.

Common pitfalls. In A, the reroot formula changes distances for all vertices, not just descendants. In C, decide whether pairs are ordered or unordered before multiplying by two; the local task uses unordered pairs.

Each task asks for every-node information on a tree.

- A first computes subtree sizes and the answer for root 1; moving the root across edge $u-v$ changes distances by $-\text{sub}[v] + (n - \text{sub}[v])$.
- B uses the diameter endpoint theorem: the farthest node from any vertex is one of the two diameter endpoints.
- C removes each node conceptually and counts pairs of endpoints lying in different resulting components.

The proof technique is contribution accounting: decide how one edge/root move changes every affected node or pair exactly once.

2 A. Sum Distances

2.1 Restatement

For every node, print the sum of distances to all other nodes.

2.2 Algorithm and proof

Correctness follows from the problem-specific reduction in the detailed guide: the generated candidates or states cover exactly the legal choices, and the final query applies the required count, minimum, maximum, merge, or classification. \square

2.3 Complexity

The complexity is the one explained in the detailed guide and follows from the number of generated states plus the searches, transitions, or graph traversals used below.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.5 Full Python solution

```
import sys
sys.setrecursionlimit(1_000_000)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; g=[[] for _ in range(n)]; idx=1
    for _ in range(n-1):
        a,b=data[idx]-1,data[idx+1]-1; idx+=2; g[a].append(b); g[b].append(a)
    sub=[1]*n; down=[0]*n
    def dfs(u,p):
        for v in g[u]:
            if v!=p: dfs(v,u); sub[u]+=sub[v]; down[u]+=down[v]+sub[v]
    ans=[0]*n
```

```

def reroot(u,p):
    for v in g[u]:
        if v!=p:
            ans[v]=ans[u]+n-2*sub[v]; reroot(v,u)
    dfs(0,-1); ans[0]=down[0]; reroot(0,-1); print(" ".join(map(str,ans)))
if __name__=="__main__": main()

```

3 B. Farthest Node Distance

3.1 Restatement

For every node, print its distance to the farthest node.

3.2 Algorithm and proof

Correctness follows from the problem-specific reduction in the detailed guide: the generated candidates or states cover exactly the legal choices, and the final query applies the required count, minimum, maximum, merge, or classification. \square

3.3 Complexity

The complexity is the one explained in the detailed guide and follows from the number of generated states plus the searches, transitions, or graph traversals used below.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<int>bfs(int s,vector<vector<int>>&g){vector<int>d(g.size(),-1);c
```

3.5 Full Python solution

```
import sys,collections
def bfs(s,g):
    d=[-1]*len(g); d[s]=0; q=collections.deque([s])
    while q:
        u=q.popleft()
        for v in g[u]:
            if d[v]<0: d[v]=d[u]+1; q.append(v)
    return d
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; g=[[] for _ in range(n)]; idx=1
    for _ in range(n-1):
```

```
    a,b=data[idx]-1,data[idx+1]-1; idx+=2; g[a].append(b); g[b].append(a)
    d0=bfs(0,g); a=max(range(n),key=d0.__getitem__); da=bfs(a,g); b=max(range(n),key=da.__ge
    print(" ".join(str(max(da[i],db[i])) for i in range(n)))
if __name__=="__main__": main()
```

4 C. Paths Through Node

4.1 Restatement

For every node, count unordered pairs of distinct nodes whose path passes through it.

4.2 Algorithm and proof

Correctness follows from the problem-specific reduction in the detailed guide: the generated candidates or states cover exactly the legal choices, and the final query applies the required count, minimum, maximum, merge, or classification. \square

4.3 Complexity

The complexity is the one explained in the detailed guide and follows from the number of generated states plus the searches, transitions, or graph traversals used below.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

4.5 Full Python solution

```
import sys
sys.setrecursionlimit(1_000_000)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; g=[] for _ in range(n)]; idx=1
    for _ in range(n-1):
        a,b=data[idx]-1,data[idx+1]-1; idx+=2; g[a].append(b); g[b].append(a)
    parent=[-1]*n; order=[0]
    for u in order:
        for v in g[u]:
            if v!=parent[u]: parent[v]=u; order.append(v)
    sub=[1]*n
```

```

for u in reversed(order[1:]): sub[parent[u]]+=sub[u]
total=n*(n-1)//2; out=[]
for u in range(n):
    bad=0
    for v in g[u]:
        s=sub[v] if parent[v]==u else n-sub[u]
        bad+=s*(s-1)//2
    out.append(str(total-bad))
print(" ".join(out))
if __name__=="__main__": main()

```

5 Testing discussion

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.